Project Workflow & Work Breakdown Structure (WBS)

> ⚠️ **Note:** This WBS is a historical task log and references some deprecated terminology (e.g., "index-based P0"). Current nomenclature uses spatially-stratified P0. See `nomenclature_migration.md` (nomenclature_migration.md).

EMS Readiness Optimization for Manhattan

Date: March 15, 2026
Version: 1.3.0
Project: EMS Readiness Optimization — Manhattan, NYC

Table of Contents

---

1. Overview

This document provides the complete Work Breakdown Structure (WBS) for the EMS Readiness Optimization project. The project follows a seven-phase methodology, progressing from problem definition through data engineering, optimization, simulation, experimentation, and final reporting. Each phase is documented with its objectives, step-by-step procedures, input/output files, scripts used, and dependencies.

Project Scope: Evaluate strategic EMS ambulance staging policies across 48 FDNY firehouses in Manhattan using discrete-event simulation, comparing spatially-stratified (P0), demand-proportional (P1), and demand-weighted MIP (P2) allocation strategies.

Total Deliverables: 14 Python modules, 23 scripts, 9 notebooks, 66+ figures, 55+ tables, 32+ documentation files.

---

2. Phase 1 — Problem Definition & Project Setup

Objective

Define the decision problem, research questions, scope, and project infrastructure.

Step-by-Step Procedure

Step	Action	Output
1.1	Define the operational decision: optimal staging of K ambulances across 48 firehouses	Problem statement
1.2	Formulate 5 research questions (RQ1–RQ5) covering spatial demand, optimal allocation, policy comparison, sensitivity, and fleet sizing	Research questions
1.3	Select primary (mean response time) and secondary (P90 (90th %ile) RT, 6-min coverage (NYC law), 8-min coverage (NFPA standard), utilization) performance measures	KPI definitions
1.4	Define system boundary: Manhattan, MVC incidents, static allocation, Haversine travel proxy	Scope statement
1.5	Document assumptions, exclusions, and limitations	Assumptions log
1.6	Set up repository structure, Python package, and configuration files	Project skeleton
1.7	Create project charter and decisions log	Governance documents

Input Files

- None (project inception)

Output Files / Deliverables

File	Description
`docs/project_charter.md`	Project charter with objectives, scope, and timeline
`docs/assumptions_log.md`	Documented assumptions (12 items) with rationale and risk
`docs/decisions_log.md`	Key decisions (DEC-001 through DEC-005)
`docs/blocker_log.md`	Blocker tracking template
`docs/source_manifest.md`	Data source inventory
`requirements.txt`	Python dependency specification
`Makefile`	Build automation targets
`README.md`	Project overview and setup instructions
`.gitignore`	Git ignore rules for large data and generated files
`src/ems_readiness/__init__.py`	Package initialization

Scripts / Notebooks Used

- Manual creation (no automated scripts in this phase)

Dependencies

- None (starting phase)

---

3. Phase 2 — Data Strategy & Engineering

Objective

Acquire, clean, and transform raw data into analysis-ready datasets; build spatial and temporal demand models; construct the firehouse-to-precinct distance matrix.

Step-by-Step Procedure

Step	Action	Output
2.1	Download raw crash records (2.24M rows), firehouse listing, precinct boundaries, and geographic boundary files from NYC Open Data	Raw data in `data/raw/`
2.2	Audit raw data quality: check boundaries, firehouse coordinates, precinct geometries, crash records	Audit reports
2.3	Spatial filtering: retain 628,811 Manhattan crashes with valid coordinates	Filtered crash dataset
2.4	Temporal extraction: derive hour, day-of-week, month from crash timestamps	Enriched crash dataset
2.5	Filter 48 Manhattan firehouses from 219 city-wide	Manhattan firehouse list
2.6	Match 30 Manhattan precincts and compute centroids	Precinct centroids
2.7	Build 48×30 Haversine distance matrix (firehouses × precincts)	Distance matrix CSV
2.8	Calibrate NHPP demand model: compute hourly factors, DOW factors, precinct proportions	Lambda tables
2.9	Build travel-time proxy model (Haversine / 20 mph with TOD factors)	Travel time module
2.10	Build service-time distribution model (LogNormal, mean=25 min, std=10 min)	Service time module
2.11	Perform exploratory data analysis (EDA): spatial, temporal, and statistical visualization	EDA figures and notebook
2.12		Model spec documents

Step	Action	Output
	Document demand model specification and service model specification	

Input Files

File	Source
`data/raw/Motor_Vehicle_Collisions_-_Crashes_20260223.csv`	NYC Open Data (536 MB, 2.24M records)
`data/raw/FDNY_Firehouse_Listing_20260223.csv`	NYC Open Data (219 firehouses)
`data/raw/Police_Precincts_20260223.csv`	NYC Open Data (78 precincts)
`data/raw/manhattan_boundary.geojson`	NYC Open Data
`data/raw/cbd_boundary.geojson`	MTA Congestion Relief Zone
`data/raw/nyc_borough_boundaries.geojson`	NYC Open Data
`data/raw/*_Data_Dictionary.xlsx`	NYC Open Data (3 data dictionaries)

Output Files / Deliverables

File	Description
`data/processed/distance_matrix_firehouse_precinct.csv`	48×30 Haversine distance matrix (miles)
`data/processed/demand_lambda_hourly.csv`	24 hourly intensity factors for NHPP
`data/processed/demand_lambda_dow.csv`	7 day-of-week intensity factors
`data/processed/demand_lambda_precinct.csv`	30 precinct demand proportions
`src/ems_readiness/utils/distance.py`	Haversine distance and matrix builder
`src/ems_readiness/service/travel_time.py`	Travel-time proxy with TOD factors
`src/ems_readiness/service/service_time.py`	LogNormal service-time sampler
`src/ems_readiness/demand/arrival_generator.py`	NHPP arrival generator (Lewis-Shedler thinning)
`configs/demand.yaml`	Demand model configuration
`configs/service.yaml`	Service model configuration
`docs/demand_model_spec.md`	Demand model specification document
`docs/service_model_spec.md`	Service & travel proxy specification
`docs/eda_and_data_split_summary.md`	EDA summary and data split documentation
`results/figures/fig_crash_heatmap.png`	Crash heatmap
`results/figures/fig_hourly_demand.png`	Hourly demand profile
`results/figures/fig_daily_demand.png`	Daily demand profile
`results/figures/fig_firehouses_map.png`	Manhattan firehouses map
`results/figures/fig_precinct_demand.png`	Precinct demand bar chart
`results/figures/fig_precinct_density.png`	Precinct density choropleth
`results/figures/fig_temporal_trends.png`	Long-term temporal trends
`results/figures/fig_hourly_rates.png`	NHPP hourly rate factors
`results/figures/fig_demand_model_fit.png`	Demand model fit diagnostics
`results/figures/distance_matrix_heatmap.png`	Distance matrix heatmap

File	Description
`results/figures/travel_time_by_tod.png`	Travel time by time-of-day
`results/figures/tod_speed_factors.png`	TOD speed factor profile
`results/figures/service_time_distribution.png`	Service time distribution
`results/figures/nhpp_arrivals_demo.png`	NHPP arrival demo

Scripts / Notebooks Used

Script / Notebook	Purpose
`scripts/data_audit.py`	Comprehensive raw data quality audit
`scripts/audit_step1_boundaries.py`	Audit geographic boundaries
`scripts/audit_step2_firehouses.py`	Audit firehouse data
`scripts/audit_step3_precincts.py`	Audit precinct data
`scripts/audit_step4_crashes.py`	Audit crash records
`scripts/demand_modeling.py`	Calibrate NHPP demand model and export lambda tables
`notebooks/02_eda_spatiotemporal.ipynb`	Exploratory data analysis with visualizations
`notebooks/03_input_modeling.ipynb`	Input modeling and demand estimation
`notebooks/04_service_travel_proxy.ipynb`	Service/travel model demonstration and validation

Dependencies

- **Depends on:** Phase 1 (project structure, configuration framework)

---

4. Phase 3 — Optimization & Policy Design

Objective

Formulate and solve MIP allocation models to generate candidate staging policies (P0, P1, P2, P2-alt, P2-cov) for simulation evaluation.

Step-by-Step Procedure

Step	Action	Output
3.1	Define P0 (spatially-stratified) baseline allocation: latitude-based firehouse selection with even spacing	Baseline policy
3.2	Implement P1 (demand-proportional) heuristic: allocate proportional to nearest-firehouse demand credit	Heuristic policy
3.3	Formulate demand-weighted MIP (P2): minimize $\Sigma\, d_j \cdot t_{ij} \cdot y_{ij}$ subject to unit total, capacity, and assignment constraints	P2 formulation
3.4	Formulate P-median model (P2-alt): select K firehouses to minimize weighted distance	P2-alt formulation
3.5	Formulate maximal coverage model (P2-cov): maximize demand within 8-minute threshold	P2-cov formulation
3.6	Implement `EMSAllocator` class as high-level solver interface	Solver module
3.7	Solve all models for $K \in \{20, 30, 40, 48\}$ and compare results	Comparison tables
3.8	Perform sensitivity analysis of objective values vs. fleet size	Sensitivity figures
3.9	Validate all policies for feasibility (unit totals, capacity caps)	Feasibility checks
3.10	Document mathematical formulations	Formulation document

Input Files

File	Source
`data/processed/distance_matrix_firehouse_precinct.csv`	Phase 2 output
`data/processed/demand_lambda_precinct.csv`	Phase 2 output
`configs/optimization.yaml`	Configuration
`configs/service.yaml`	Travel speed configuration

Output Files / Deliverables

File	Description
`src/ems_readiness/optimization/models.py`	MIP formulations (demand-weighted, p-median, maximal coverage)
`src/ems_readiness/optimization/policies.py`	Baseline policies (uniform, demand-proportional)
`src/ems_readiness/optimization/allocator.py`	High-level `EMSAllocator` solver class
`configs/optimization.yaml`	Optimization configuration
`docs/optimization_formulation.md`	Mathematical specification of all models
`docs/optimization_results.md`	Optimization results and comparison
`results/tables/optimization_comparison.csv`	Model comparison table
`results/figures/opt_allocation_comparison.png`	Allocation comparison visualization
`results/figures/opt_inputs.png`	Optimization inputs visualization
`results/figures/opt_sensitivity.png`	Sensitivity analysis figure

Scripts / Notebooks Used

Script / Notebook	Purpose
`scripts/run_optimization_comparison.py`	Run all optimization models and generate comparison
`notebooks/05_optimization.ipynb`	Interactive optimization demo and analysis
`notebooks/05_optimization.py`	Source script for optimization notebook

Dependencies

- **Depends on:** Phase 2 (distance matrix, demand data, travel time model)

---

5. Phase 4 — Simulation Design & Implementation

Objective

Design the conceptual DES model and implement a modular simulation engine in Python using SimPy.

Step-by-Step Procedure

Step	Action	Output
4.1	Define entities (Incident), resources (EMSUnit), queues (FIFO), and state variables	Conceptual model
4.2	Specify 5 event types: Arrival, Dispatch, Service Start, Service Completion, End of Simulation	Event logic
4.3	Design nearest-available dispatch logic with Haversine travel-time estimation	Dispatch specification
4.4	Define performance measures: mean RT, P90 (90th %ile) RT, 6-min coverage (NYC law), 8-min coverage (NFPA standard), utilization, queue metrics	KPI specification
4.5	Document random phenomena: NHPP arrivals (Stream 1), precinct assignment (Stream 2), service times (Stream 3)	RNG strategy
4.6	Create event flow diagram and single-incident timeline	Flowcharts
4.7	Implement `Incident` dataclass with full lifecycle tracking	`entities.py`
4.8	Implement `EMSUnit`, `UnitPool`, and `UnitStatus` for resource management	`resources.py`
4.9	Implement `NearestAvailableDispatcher` with cached travel-time matrices	`dispatcher.py`
4.10	Implement `MetricsCollector` for KPI tracking	`metrics.py`
4.11		`engine.py`

Step	Action	Output
	Implement `EMSSimulation` main engine with SimPy process-interaction	
4.12	Implement `BatchRunner` for replicated experiments with CRN support	`runner.py`
4.13	Write unit tests (39 tests across 4 modules)	Test suite
4.14	Document conceptual model and code architecture	Documentation

Input Files

File	Source
`data/processed/distance_matrix_firehouse_precinct.csv`	Phase 2 output
`data/processed/demand_lambda_*.csv`	Phase 2 output (3 lambda tables)
`configs/simulation.yaml`	Simulation configuration
`configs/demand.yaml`	Demand model parameters
`configs/service.yaml`	Service model parameters

Output Files / Deliverables

File	Description
`src/ems_readiness/simulation/__init__.py`	Simulation package initialization
`src/ems_readiness/simulation/entities.py`	`Incident` dataclass
`src/ems_readiness/simulation/resources.py`	`EMSUnit`, `UnitPool`, `UnitStatus`
`src/ems_readiness/simulation/dispatcher.py`	`NearestAvailableDispatcher`
`src/ems_readiness/simulation/metrics.py`	`MetricsCollector`
`src/ems_readiness/simulation/engine.py`	`EMSSimulation` main DES engine
`src/ems_readiness/simulation/runner.py`	`BatchRunner` for experiment execution
`configs/simulation.yaml`	DES engine configuration
`docs/conceptual_model.md`	Complete conceptual model specification
`docs/code_documentation.md`	Code architecture and module documentation
`tests/`	39 unit tests across 4 test modules

Scripts / Notebooks Used

Script / Notebook	Purpose
`notebooks/06_simulation_debug.ipynb`	Simulation debugging and validation
`notebooks/06_simulation_debug.py`	Source script for simulation notebook

Dependencies

- **Depends on:** Phase 2 (demand model, distance matrix), Phase 3 (allocation policies)

---

6. Phase 5 — Verification & Validation

Objective

Verify that the simulation code correctly implements the conceptual model and validate that the model produces credible results.

Step-by-Step Procedure

Step	Action	Output
5.1	**Verification Test 1 — Toy Example:** Run a small deterministic scenario with known analytical solution; hand-trace event timeline	Toy timeline figure
5.2	**Verification Test 2 — Zero Demand:** Set $\lambda_0 = 0$ and verify no incidents are generated	Zero-demand check
5.3	**Verification Test 3 — Single-Unit Saturation:** Run with K=1 under high demand; verify queueing occurs correctly	Saturation test
5.4	**Verification Test 4 — Extreme Demand:** Stress test under 10× demand to verify queue accumulation and system stability	Extreme demand test
5.5	**Validation Pilot 1 — Directional:** Compare P0 vs. P2 and confirm P2 dominates on mean RT	Directional validation
5.6	**Validation Pilot 2 — Fleet Sensitivity:** Verify response time decreases monotonically with increasing K	Monotonicity check
5.7	**Validation Pilot 3 — Demand Sensitivity:** Verify response time increases with demand intensity	Demand response check
5.8	Run all 39 unit tests and confirm 100% pass rate	Test results
5.9	Document all V&V findings, corrections, and acceptance criteria	V&V log

Input Files

File	Source
All Phase 2 processed data	Phase 2 output
All Phase 3 allocation policies	Phase 3 output
`configs/simulation.yaml`	Simulation configuration

Output Files / Deliverables

File	Description
`docs/verification_log.md`	Complete V&V results documentation
`results/figures/verification_toy_timeline.png`	Toy example event timeline
`results/figures/validation_p0_vs_p2.png`	Validation: P0 vs. P2 comparison
`results/figures/validation_sensitivity_K.png`	Validation: Fleet sensitivity
`results/figures/validation_sensitivity_demand.png`	Validation: Demand sensitivity

Scripts / Notebooks Used

Script / Notebook	Purpose
`scripts/run_verification.py`	Execute all 4 verification tests (seed=42)
`scripts/run_validation_pilots.py`	Execute all 3 validation pilots

Dependencies

- **Depends on:** Phase 4 (simulation engine), Phase 3 (allocation policies), Phase 2 (data)

---

7. Phase 6 — Experimentation & Analysis

Objective

Execute production simulation experiments, perform statistical analysis, and generate publication-quality results.

Step-by-Step Procedure

Step	Action	Output
6.1	Freeze experimental design: 4 main experiments + CBD robustness	Experiment matrix
6.2	**Exp 1 — Policy Comparison:** 3 policies × 30 replications = 90 runs at K=20	Exp1 results
6.3	**Exp 2 — Fleet Sensitivity:** 3 policies × 6 fleet sizes × 30 reps = 540 runs	Exp2 results
6.4	**Exp 3 — Demand Sensitivity:** 3 policies × 6 demand multipliers × 30 reps = 540 runs	Exp3 results
6.5	**Exp 4 — Service Robustness:** 3 policies × 3 service times × 30 reps = 270 runs	Exp4 results
6.6	Execute all 1,440 production runs	Raw simulation output
6.7	**CBD Experiment:** 11 CBD scenarios × 30 replications = 330 additional runs	CBD results
6.8	Compute descriptive statistics, ANOVA, Tukey HSD, effect sizes, and CIs	Statistical tables
6.9	Analyze queue metrics across all 1,770 runs	Queue analysis
6.10	Analyze seasonal variation patterns	Seasonal analysis
6.11	Generate publication-quality figures (5 main figures)	Publication figures
6.12	Generate project summary dashboard	Dashboard figure
6.13	Document experimental design and output analysis	Design document

Input Files

File	Source
All processed data from Phase 2	Phase 2 output
Allocation policies from Phase 3	Phase 3 output
Verified simulation engine from Phase 4/5	Phase 4–5 output
`configs/simulation.yaml`	Experiment configuration
`configs/cbd_scenario.yaml`	CBD experiment configuration

Output Files / Deliverables

File	Description
`results/tables/descriptive_statistics.csv`	Full descriptive statistics
`results/tables/anova_results.csv`	ANOVA results
`results/tables/posthoc_comparisons.csv`	Tukey HSD post-hoc tests
`results/tables/confidence_intervals.csv`	95% confidence intervals
`results/tables/effect_sizes.csv`	Cohen’s d effect sizes
`results/tables/exp1_summary.csv`	Experiment 1 summary
`results/tables/exp2_pivot_rt.csv`	Experiment 2 pivot table
`results/tables/exp3_pivot_rt.csv`	Experiment 3 pivot table
`results/tables/exp4_pivot_rt.csv`	Experiment 4 pivot table
`results/tables/sensitivity_summary.csv`	Overall sensitivity summary
`results/tables/cbd_comparison.csv`	CBD comparison table
`results/tables/cbd_summary_all.csv`	CBD full summary
`results/tables/queue_statistics.csv`	Queue statistics
`results/tables/queue_anova.csv`	Queue ANOVA results
`results/tables/seasonal_analysis.csv`	Seasonal variation analysis
`results/tables/table1–4_*.csv` and `.tex`	Publication tables (CSV + LaTeX)
`results/figures/exp1–4_*.png`	Experiment result figures
`results/figures/pub_fig1–5_*.png`	Publication-quality figures
`results/figures/queue_*.png`	Queue analysis figures
`results/figures/seasonal_*.png`	Seasonal analysis figures
`results/figures/project_summary_dashboard.png`	Summary dashboard
`docs/experimental_design.md`	Experimental design specification
`docs/output_analysis.md`	Output analysis documentation
`docs/cbd_robustness_analysis.md`	CBD robustness analysis report

File	Description
`docs/queue_analysis.md`	Queue analysis report
`docs/gap_closure_report.md`	Gap closure report

Scripts / Notebooks Used

Script / Notebook	Purpose
`scripts/run_production_experiments.py`	Execute 1,440 production runs (SEED_BASE=42)
`scripts/run_cbd_experiment.py`	Execute 330 CBD robustness runs (SEED_BASE=42)
`scripts/analyze_production_results.py`	Statistical analysis of production results
`scripts/analyze_queue_metrics.py`	Queue metrics analysis
`scripts/analyze_seasonal_patterns.py`	Seasonal variation analysis
`scripts/generate_publication_figures.py`	Generate publication-quality figures
`scripts/generate_summary_dashboard.py`	Generate summary dashboard
`notebooks/07_production_results.ipynb`	Interactive production results exploration
`notebooks/08_statistical_analysis.ipynb`	Interactive statistical analysis
`notebooks/09_cbd_analysis.ipynb`	CBD robustness analysis notebook

Dependencies

- **Depends on:** Phase 5 (verified/validated simulator), all previous phases

---

8. Phase 7 — Reporting & Final Delivery

Objective

Compile all findings into a full technical report, executive presentation, and complete project archive.

Step-by-Step Procedure

Step	Action	Output
7.1	Write the full technical report covering all sections (Introduction through Appendices)	Technical report
7.2	Add formal abstract (~250 words) and executive summary	Report front matter
7.3	Create List of Figures (44 figures) and List of Tables (24 tables)	Report indices
7.4	Add “Why DES?” justification section	Methodology rationale
7.5	Embed inline conceptual model flowchart	Visual documentation
7.6	Add reproducibility section (seeds, environment, instructions)	Reproducibility guide
7.7	Write executive presentation with key findings and recommendations	Presentation document
7.8	Prepare implementation roadmap (0–3, 3–6, 6–12, 12+ months)	Roadmap
7.9	Create file inventory and project archive documentation	Archive manifest
7.10	Perform compliance assessment	Compliance report
7.11	Address any remaining gaps identified in assessment	Gap remediation
7.12	Generate PDF versions of all documentation	PDF deliverables
7.13	Create Work Breakdown Structure documentation	This document
7.14	Final review and version tagging	Release

Input Files

File	Source
All results from Phases 1–6	Previous phases
All documentation from Phases 1–6	Previous phases

Output Files / Deliverables

File	Description
`docs/technical_report.md`	Comprehensive technical report (fully compliant)
`docs/technical_report.pdf`	PDF version of technical report
`docs/executive_summary.md`	Standalone executive summary
`docs/executive_presentation.md`	Executive presentation
`docs/implementation_roadmap.md`	Implementation roadmap
`docs/final_summary.md`	Final project summary
`docs/file_inventory.md`	Complete file inventory
`docs/project_archive.md`	Archive documentation
`docs/phase21_assessment.md`	compliance assessment
`docs/gap_remediation_plan.md`	Gap remediation plan
`docs/phase_comparison_summary.md`	Phase comparison summary
`docs/project_workflow_wbs.md`	This WBS document
`docs/cbd_comparison_and_validity_report.md`	CBD comparison and validity report
`docs/cbd_definition.md`	CBD definition and boundary documentation
All PDF versions of above documents	Generated PDFs
`README.md` (updated)	Final project README

Scripts / Notebooks Used

Script / Notebook	Purpose
Manual writing and compilation	Documentation authoring
PDF generation pipeline	Markdown → PDF conversion

Dependencies

- **Depends on:** All previous phases (1–6)

---

9. Phase 8 — Alternative Analyses & Extensions

Objective

Conduct alternative analyses to test the robustness of modelling choices and compare Manhattan-wide vs. CBD-focused optimization strategies.

Step-by-Step Procedure

Step	Action	Output
8.1	Implement Manhattan (taxicab) distance metric in `distance.py` alongside Haversine	Updated distance module
8.2	Generate 48×30 Manhattan distance matrix from firehouse-to-precinct centroids	Manhattan distance matrix CSV
8.3	Run distance metric comparison experiment: solve P2 with both metrics, simulate, compare RT	Distance comparison results
8.4	Generate distance comparison figures (heatmap, scatter, bar, boxplot)	4 comparison figures
8.5	Implement CBD-focused optimization models: `build_cbd_focused_demand_weighted()` and `build_cbd_focused_coverage()`	Updated models module
8.6	Run CBD-focused vs. Manhattan-wide comparison experiment: solve, simulate, compare RT by zone	CBD comparison results
8.7	Generate CBD comparison figures (bar chart, allocation map, equity trade-off)	3 comparison figures
8.8	Document distance metric comparison findings	Distance metric report
8.9	Document CBD-focused optimization findings	CBD optimization report
8.10	Write alternative analyses summary integrating both studies	Summary report
8.11	Update technical report (§5.10, §5.11), final summary,	Updated docs

Step	Action	Output
	WBS, README, decisions log, assumptions log	

Input Files

File	Source
`data/processed/distance_matrix_firehouse_precinct.csv`	Phase 2 output (Haversine)
`data/processed/demand_lambda_precinct.csv`	Phase 2 output
`data/raw/Police_Precincts_20260223.csv`	Raw precinct geometries
`data/raw/cbd_boundary.geojson`	CBD boundary
`configs/optimization.yaml`	Optimization config
`configs/service.yaml`	Travel speed config

Output Files / Deliverables

File	Description
`data/processed/distance_matrix_firehouse_precinct_manhattan.csv`	48×30 Manhattan distance matrix (miles)
`results/distance_comparison/comparison_table.csv`	Haversine vs. Manhattan metric comparison
`results/distance_comparison/allocation_comparison.csv`	Side-by-side allocations
`results/distance_comparison/distance_matrices_heatmap.png`	Dual-panel heatmap
`results/distance_comparison/distance_scatter.png`	Scatter: Manhattan vs. Haversine
`results/distance_comparison/distance_comparison_bar.png`	RT comparison bar chart
`results/distance_comparison/distance_comparison_boxplot.png`	RT distribution boxplot
`results/cbd_focused_comparison/comparison_table.csv`	CBD-focused vs. Manhattan-wide metrics
`results/cbd_focused_comparison/allocations.csv`	Side-by-side allocations
`results/cbd_focused_comparison/cbd_focused_comparison.png`	RT comparison bar chart
`results/cbd_focused_comparison/allocation_comparison.png`	Allocation map comparison
`results/cbd_focused_comparison/equity_tradeoff.png`	CBD vs. non-CBD equity trade-off
`docs/distance_metric_comparison.md`	Distance metric comparison report
`docs/cbd_focused_optimization_analysis.md`	CBD-focused optimization report
`docs/alternative_analyses_summary.md`	Combined alternative analyses summary

Scripts / Notebooks Used

Script / Notebook	Purpose
`scripts/generate_manhattan_distance_matrix.py`	Generate Manhattan distance matrix
`scripts/run_distance_comparison_experiment.py`	Run Haversine vs. Manhattan comparison (P2 + simulation)
`scripts/run_cbd_focused_optimization.py`	Run CBD-focused vs. Manhattan-wide comparison

Dependencies

- **Depends on:** Phase 2 (distance matrix, demand data), Phase 3 (optimization models), Phase 4–5 (simulation engine)

---

10. Phase 9 — Capacity & Baseline Improvements

Objective

Conduct full capacity sensitivity analysis (cap 1–5), implement spatially-stratified P0 baseline, and re-run all production experiments as Extended Fleet Analysis with updated defaults (capacity=2, P0 (spatially-stratified)).

Step-by-Step Procedure

Step	Action	Output
9.1	Implement capacity parameter in optimization models: update `build_p_median` for capacity-aware allocation; add `solve_model()` convenience function	Updated models module
9.2	Run initial capacity comparison: cap=2 vs cap=5 for K ∈ {20, 40} with P0, P1, P2	Initial comparison results
9.3	Run full-spectrum capacity sensitivity: cap ∈ {1, 2, 3, 5, unlimited} × K ∈ {10, 15, 20, 25, 30, 35, 40, 45, 48}	450 optimization runs
9.4	Generate capacity sensitivity figures and tables	Capacity comparison visualizations
9.5	Document capacity sensitivity findings	Capacity analysis reports
9.6	Implement spatially-stratified allocation: `spatially_stratified_allocation()` with latitude, grid, and maximin methods	Updated policies module
9.7	Implement `spatial_stratification_analysis()` for comparing stratification methods	Analysis function
9.8	Run P0 (spatially-stratified) analysis: evaluate latitude-based stratification vs index-based P0	P0 spatial comparison
9.9	Update `configs/optimization.yaml` default capacity from 5 to 2	Updated config
9.10	Run Extended Fleet Analysis experiments: 9 fleet sizes × 3 policies × 30 replications =	Extended Fleet Analysis results

Step	Action	Output
	810 runs (cap=2, P0 (spatially-stratified))	
9.11	Update all documentation: technical report, README, WBS, file inventory, decisions log, assumptions log, code docs, final summary	Updated documentation (v1.3.0)

Input Files

File	Source
`data/processed/distance_matrix_firehouse_precinct.csv`	Phase 2 output
`data/processed/demand_lambda_precinct.csv`	Phase 2 output
`data/processed/firehouses_manhattan.csv`	Phase 2 output
`configs/optimization.yaml`	Configuration (updated cap=2)
`configs/service.yaml`	Travel speed configuration
`configs/simulation.yaml`	Simulation configuration

Output Files / Deliverables

File	Description
`results/capacity_comparison/`	60+ files: allocations, figures, tables for cap 1–5 sensitivity
`results/production_v2/`	Complete analysis results: allocations, simulation data, figures, tables
`docs/firehouse_capacity_analysis.md`	Firehouse capacity methodology and initial analysis
`docs/capacity_sensitivity_analysis.md`	Full-spectrum capacity sensitivity report
`src/ems_readiness/optimization/policies.py`	Updated with `spatially_stratified_allocation()`
`src/ems_readiness/optimization/models.py`	Updated `build_p_median` + `solve_model()`
`configs/optimization.yaml`	Default capacity updated to 2

Scripts / Notebooks Used

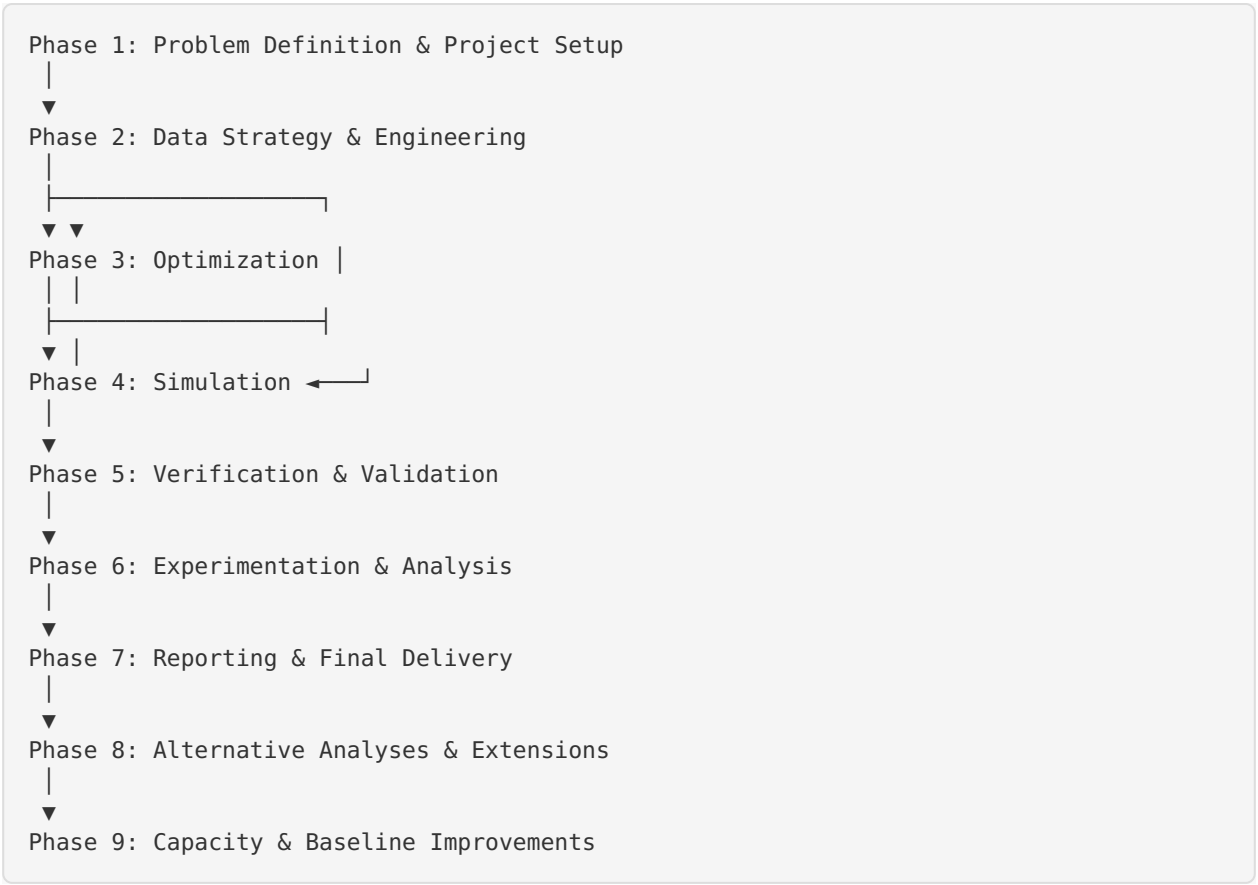
Script / Notebook	Purpose
`scripts/capacity_sensitivity_analysis.py`	Initial cap=2 vs cap=5 comparison
`scripts/capacity_sensitivity_full_spectrum.py`	Full-spectrum cap 1–5 sensitivity (450 runs)
`scripts/p0_spatial_analysis.py`	Spatial stratification analysis and visualization
`scripts/run_production_v2.py`	Extended Fleet Analysis experiment runner (810 runs)

Dependencies

- **Depends on:** Phase 2 (distance matrix, demand data), Phase 3 (optimization models), Phase 4–5 (simulation engine), Phase 6 (initial production results for comparison)

---

11. Phase Dependency Diagram



Critical Path: Phase 1 → Phase 2 → Phase 4 → Phase 5 → Phase 6 → Phase 7 → Phase 8 → Phase 9

Parallel Opportunities:
- Phase 3 (Optimization) and Phase 4 (Simulation design) can partially overlap, as the conceptual

model design does not depend on optimization results. However, the simulation implementation requires the allocation policies from Phase 3 as input.
- Documentation tasks in Phase 7 can begin concurrently with Phase 6 experimentation.
- Phase 8 can begin once Phase 6 is complete, and runs in parallel with or after Phase 7.
- Phase 9 extends Phase 8 with capacity analysis and baseline improvements, requiring all prior phases.

---

12. Timeline Summary

Phase	Description	Estimated Duration	Cumulative
1	Problem Definition & Setup	1 week	Week 1
2	Data Strategy & Engineering	2 weeks	Week 3
3	Optimization & Policy Design	1 week	Week 4
4	Simulation Design & Implementation	2 weeks	Week 6
5	Verification & Validation	1 week	Week 7
6	Experimentation & Analysis	2 weeks	Week 9
7	Reporting & Final Delivery	2 weeks	Week 11
8	Alternative Analyses & Extensions	1 week	Week 12
9	Capacity & Baseline Improvements	1 week	Week 13

Total Project Duration: ~13 weeks

Key Milestones

Milestone	Target	Deliverable
M1: Data Ready	Week 3	Processed datasets, distance matrix, lambda tables
M2: Policies Generated	Week 4	P0, P1, P2 allocation vectors for all K values
M3: Simulator Operational	Week 6	Working DES engine with all components
M4: V&V Complete	Week 7	Verified and validated simulation model
M5: Experiments Complete	Week 9	1,770 production runs with statistical analysis
M6: Final Submission	Week 11	Complete report, presentation, and reproducible archive
M7: Alternative Analyses	Week 12	Distance metric & CBD-focused comparison reports
M8: Capacity & Baseline	Week 13	Capacity sensitivity (cap 1–5), P0 (spatially-stratified), Extended Fleet Analysis (810 runs)

Appendix: Complete Script Inventory

Script	Phase	Purpose	Key Parameters
`scripts/data_audit.py`	2	Raw data quality audit	—
`scripts/audit_step1_boundaries.py`	2	Boundary file audit	—
`scripts/audit_step2_firehouses.py`	2	Firehouse data audit	—
`scripts/audit_step3_precincts.py`	2	Precinct data audit	—
`scripts/audit_step4_crashes.py`	2	Crash records audit	—
`scripts/demand_modeling.py`	2	NHPP demand calibration	base_rate=3.48
`scripts/run_optimization_comparison.py`	3	Run all optimization models	K∈{20,30,40,48}
`scripts/run_verification.py`	5	4 verification tests	seed=42
`scripts/run_validation_pilots.py`	5	3 validation pilots	seed_base=100,200,300
`scripts/run_production_experiments.py`	6	1,440 production runs	SEED_BASE=42, R=30
`scripts/run_cbd_experiment.py`	6	330 CBD robustness runs	SEED_BASE=42, R=30
`scripts/analyze_production_results.py`	6	Statistical analysis	—
	6	Queue metrics analysis	—

Script	Phase	Purpose	Key Parameters
`scripts/analyze_queue_metrics.py`			
`scripts/analyze_seasonal_patterns.py`	6	Seasonal variation analysis	—
`scripts/generate_publication_figures.py`	6	Publication figures	—
`scripts/generate_summary_dashboard.py`	6	Summary dashboard	—
`scripts/generate_manhattan_distance_matrix.py`	8	Manhattan distance matrix	—
`scripts/run_distance_comparison_experiment.py`	8	Haversine vs. Manhattan comparison	K=20, reps=10
`scripts/run_cbd_focused_optimization.py`	8	CBD-focused vs. Manhattan-wide	K=20, reps=10
`scripts/capacity_sensitivity_analysis.py`	9	Initial cap=2 vs cap=5 comparison	K∈{20,40}
`scripts/capacity_sensitivity_full_spectrum.py`	9	Full-spectrum cap 1–5 sensitivity	cap∈{1,2,3,5,∞}, K∈{10–48}
`scripts/run_p0_spatial_analysis.py`	9	P0 spatial stratification analysis	latitude, grid, maximin
`scripts/run_production_v2.py`	9	Extended Fleet Analysis experiments	cap=2, P0 (spatially-stratified), 810 runs

Part of the EMS Readiness Optimization project, Phase 9.
Version 1.3.0 — March 15, 2026